🐳 Section 3 — Running Your First Container

A clean, minimal introduction that demonstrates Docker in action while illustrating OS concepts such as isolation, filesystem views, and process namespaces.

🔧 3.1 Installing Docker (Minimal Notes)

Linux:

```
sudo apt-get update
sudo apt-get install docker.io
sudo systemctl enable --now docker
```

macOS / Windows:

Install *Docker Desktop* from docker.com.

▶️ 3.2 Running a Basic Container

The simplest possible container run:

```
docker run hello-world
```

Behavior:

- Docker checks for the image locally.
- If missing, it pulls the image.
- Runs a small executable inside an isolated filesystem + namespace environment.

This demonstrates the standard container lifecycle: *pull* → *create* → *run* → *exit*.

🐧 3.3 Launching an Interactive Linux Environment

Run a full Ubuntu environment:

```
docker run -it ubuntu bash
```

Inside the Container:

You now have:

- a new process namespace
- a new PID 1
- a new mount namespace
- isolated networking
- its own `/bin`, `/usr`, `/lib`, etc.

Commands to try inside:

🌲 3.4 Observing Isolation From the Host

Run in one terminal:

```
docker run -it ubuntu bash
```

In a second host terminal:

```
ps aux | grep bash
```

You will see:

- a **bash** process on the host (because containers run as host processes),
- but inside the container, that same process appears as **PID 1**.

This demonstrates that containers are *real host processes* with *virtualized views* of system resources.

📦 3.5 Running Background Containers

```
docker run -d nginx
```

This:

- starts an NGINX web server in the background
- returns a container ID
- exposes nothing to the host yet (no mapped ports)

Check running containers:

```
docker ps
```

Stop the container:

```
docker stop <id>
```

📁 3.6 Inspecting the Container Filesystem

To explore without running interactively:

```
docker run --rm -it ubuntu sh -c "ls -R /"
```

Or inspect an existing container:

```
docker inspect <id>
```

Relevant OS-related fields:

- **Namespaces** (PIDs, mounts, networks)
- **Cgroups** (resource restrictions)
- **Filesystem layers**

This reinforces how Docker composes OS primitives into a container environment.

🧪 3.7 Minimal Experiment for OS Courses

Experiment: Compare host vs. container process IDs.

Host:

```
echo "Host PID:" $$
```

Container:

```
docker run --rm -it ubuntu bash -c 'echo "Container PID:" $$; sleep 2'
```

You'll see:

- The container PID is reported as `1` inside the namespace.
- The host PID is a normal high integer.

This demonstrates that PID namespaces create *independent process hierarchies* even

💡 Summary

Running a container illuminates key OS mechanisms:

- Process isolation via namespaces
- Filesystem isolation via mount namespaces + image layering
- Networking isolation via virtual interfaces
- Host scheduling of container processes
- Fast, reproducible execution environments